

L'Académie de Sorcellerie

Cahier des charges — Projet 4 jours • Flask • SQLAlchemy • API REST

Niveau junior / médior

Contexte

Votre équipe est mandatée par le Conseil des Enseignants pour construire l'API qui gèrera l'ensemble de la vie académique de l'Académie : élèves, maisons, cours, examens, compétences magiques, tournois inter-maisons et passage de fin d'année. Le cahier des charges décrit ce que l'API doit permettre de faire — pas comment l'implémenter. L'organisation du code, la structure des fichiers et les choix d'architecture sont entre vos mains.

Ce projet est volontairement dense : chaque jour représente une journée complète de travail (6 à 7h) et s'appuie sur ce qui a été construit la veille. Ne sous-estimez pas le temps de conception du modèle de données en jour 1 — une bonne base facilite tout le reste de la semaine.

À noter — sécurité hors scope, mais pas absente

Contraintes techniques

- Python 3 • Flask pour l'API • Flask-SQLAlchemy pour l'accès aux données.
- Base de données au choix (SQLite pour la simplicité, ou toute autre base relationnelle supportée par SQLAlchemy).
- Aucune contrainte sur l'organisation des fichiers, le style d'architecture (app factory, blueprints, monolithe simple...) ou les bibliothèques annexes — à vous de choisir et de justifier si on vous le demande.
- Un commit git par jour minimum, avec un message clair. L'historique fait partie de l'évaluation.

JOUR 1 — Fondations et comptes

Modèle de données de base, CRUD, connexion simulée

Objectif

Poser un modèle de données solide couvrant toute la structure de l'Académie, livrer le CRUD sur les ressources de base, et mettre en place le mécanisme de connexion simulée qui servira toute la semaine.

Entités attendues

- **Maison** : nom, couleur, fondateur, valeurs.
- **Professeur** : nom, matière enseignée, ancienneté.
- **Cours** : intitulé, niveau requis, capacité maximale, professeur responsable, année académique.
- **Élève** : nom, année d'études (1 à 7), maison d'appartenance, familial, statut (actif / diplômé / renvoyé).
- **Utilisateur** : email, mot de passe (stocké en clair pour l'instant — annotez-le clairement comme temporaire dans le code), rôle (élève / professeur / admin), et un lien optionnel vers un Élève ou un Professeur selon le rôle.

Relations minimales

- Une **Maison** possède plusieurs **Élèves** (1-N).
- Un **Professeur** est responsable de plusieurs **Cours** (1-N).
- Un **Utilisateur** de rôle élève est lié à exactement un **Élève** ; un utilisateur admin n'est lié à personne. Réfléchissez à comment contraindre ça proprement (contrainte applicative, pas forcément en base).

Connexion simulée

- **POST /login** : reçoit email + mot de passe, vérifie la correspondance (comparaison simple, pas de hachage à ce stade), renvoie l'utilisateur, son rôle et l'identifiant lié (eleve_id ou professeur_id).
- Tous les endpoints qui suivront cette semaine et qui distinguent "espace élève" / "espace admin" liront un header **X-User-Id** pour savoir qui fait la requête, et se comporteront comme si c'était vérifié (même si rien n'empêche aujourd'hui de mettre n'importe quel id).
- Un appel sans header X-User-Id sur un endpoint qui en a besoin doit renvoyer une erreur explicite (401 ou 400 selon votre convention), pas un comportement silencieux.

Exigences fonctionnelles

- CRUD complet sur Maison, Professeur, Cours, Élève.
- Réponses JSON cohérentes, codes HTTP appropriés (200, 201, 400, 404).
- Script de seed : au moins 4 maisons, une trentaine d'élèves répartis sur les 7 années, quelques professeurs et cours, et un utilisateur par élève/professeur plus un compte admin.

Critère de fin de journée

Une personne extérieure peut créer, lire, modifier et supprimer chaque ressource, se "connecter" avec un compte de test et obtenir son rôle et son id en retour.

JOUR 2 — Vie académique

Inscriptions, examens, résultats, premier endpoint métier

Objectif

Introduire la relation N-N enrichie entre élèves et cours, ajouter les examens et leurs résultats, et livrer le premier endpoint métier qui automatise une vraie décision pédagogique.

Nouvelles entités et relations

- **Inscription** (association enrichie Élève ↔ Cours) : date d'inscription, statut (inscrit / en cours / validé / abandonné).
- **Examen** : rattaché à un Cours (1-N), avec un titre, une date et un **seuil de réussite**.
- **Résultat** (association enrichie Élève ↔ Examen) : note obtenue.
- Un **Cours** ne peut pas accepter plus d'élèves que sa capacité maximale : l'inscription doit être refusée proprement si le cours est complet.

Endpoints à ajouter

- **Inscrire un élève à un cours** (en respectant la capacité maximale).
- **Saisir les résultats d'un examen en masse** (un payload avec la liste des notes par élève, pas un appel par élève).
- **Clôturer un examen** : endpoint métier qui, une fois les résultats saisis, met à jour le statut des inscriptions concernées (validé si la moyenne du cours est atteinte, par exemple) et calcule la moyenne du cours.

Espace élève / espace admin

- Élève (via X-User-Id) : "mes cours", "mes notes", "mon dossier" — un élève ne doit voir que ses propres données.
- Admin : liste filtrable des résultats par cours ou par examen, moyenne par cours.

Chasse au N+1 (obligatoire)

Quand on liste les élèves d'un cours et qu'on accède à **eleve.maison.nom** pour chacun dans une boucle (ou l'inverse : lister les cours d'un élève), SQLAlchemy en chargement paresseux exécute une requête par élément de la boucle.

- Activez le log SQL (**SQLALCHEMY_ECHO = True**), comptez les requêtes sur au moins un endpoint de listing de ce jour.

- Corrigez avec **joinedload()** ou **selectinload()**, en rendant le chargement optionnel via un paramètre de requête plutôt que systématique.
- Documentez la différence avant/après (nombre de requêtes) dans le README ou un fichier PERFORMANCE.md.

Tests automatisés

- Au moins 2 tests métier : la clôture d'un examen met bien à jour le statut des inscriptions selon le seuil, et un élève ne peut pas s'inscrire à un cours complet.

Critère de fin de journée

La relation N-N (élève ↔ cours) est vérifiable sans duplication. La clôture d'un examen produit un résultat cohérent et vérifiable, pas un code 200 vide. Vous avez un chiffre précis de requêtes SQL avant/après correction du N+1.

JOUR 3 — Compétences et tournois

Catalogue de compétences, déblocage automatique, duels inter-maisons

Objectif

Ajouter un système de compétences magiques débloquables selon les résultats scolaires ou les performances en tournoi, et un système de tournois avec duels entre élèves.

Compétences

- **Compétence** : catalogue large (au moins 15 à 20 entrées dans le seed), avec nom, catégorie (sorts offensifs, défensifs, potions, divination, métamorphose...), description, et une condition de déblocage : liée à un examen précis avec une note minimale, ou liée à une victoire en tournoi.
- **Maîtrise** (association enrichie Élève ↔ Compétence) : date d'obtention, source (examen ou tournoi) et référence vers cette source.

Tournois et duels

- **Tournoi** : nom, année, maison organisatrice (facultatif).
- **Duel** : rattaché à un tournoi, oppose deux élèves, enregistre un vainqueur.

Endpoints métier

- **Évaluer les compétences débloquées** après la clôture d'un examen : pour chaque élève ayant atteint le seuil requis par une compétence liée à cet examen, créer automatiquement la Maîtrise correspondante si elle n'existe pas déjà. Cette opération doit être **idempotente** : la relancer deux fois ne doit jamais créer de doublon.
- **Clôturer un tournoi** : détermine le vainqueur global (par exemple le plus de victoires en duel), débloque la ou les compétences associées à la victoire, et ajoute un score de réputation à la maison du vainqueur.

Espace élève / espace admin

- Élève : "mes compétences débloquées", "mon historique de tournois et de duels".
- Admin : gérer le catalogue de compétences, créer un tournoi, enregistrer les duels au fur et à mesure.

Pagination et filtres

- Listing des compétences filtrable par catégorie ; listing des tournois filtrable par année ; pagination sur les deux.

Tests automatisés

- Un test qui vérifie qu'un élève repassant au-dessus du même seuil ne débloque pas deux fois la même compétence (idempotence).

- Un test qui vérifie qu'à la clôture d'un tournoi, le vainqueur obtient bien la compétence associée et que la réputation de sa maison augmente.

Critère de fin de journée

On peut clôturer un examen, voir les compétences se débloquent automatiquement pour les élèves concernés, créer un tournoi, enregistrer des duels et le clôturer en voyant la réputation de maison évoluer — le tout vérifiable via l'espace élève et l'espace admin.

JOUR 4 — Passage de fin d'année

Endpoint métier majeur, validation stricte, fiabilisation

Objectif

Livrer l'endpoint métier le plus complexe du projet — le passage de fin d'année — et fiabiliser l'ensemble de l'API construite cette semaine.

Clôturer l'année académique

Endpoint métier qui, pour chaque élève actif, exécute la logique suivante :

- calcule la **moyenne générale** de l'élève sur l'ensemble de ses inscriptions validées de l'année,
- si la moyenne dépasse un seuil configurable et que l'élève n'est pas en année 7 : **promotion** à l'année suivante,
- si la moyenne est insuffisante : **redoublement** (l'élève reste dans la même année),
- si l'élève est en année 7 avec une moyenne suffisante : **diplomation** — l'élève passe en statut diplômé et est archivé (pas supprimé : son dossier, ses compétences et son historique de tournois restent consultables).
- L'opération doit être protégée contre un rejeu accidentel : relancer la clôture d'une année déjà clôturée doit être refusé explicitement, pas silencieusement ré-exécuté.

Validation stricte des payloads

- Introduisez une bibliothèque de validation de schéma (marshmallow, pydantic, ou équivalent) sur l'ensemble des endpoints d'écriture ajoutés cette semaine.
- Un payload invalide (type incorrect, champ manquant, valeur hors limites — ex. une note négative ou supérieure au maximum) doit renvoyer un 400 avec un message qui indique précisément quel champ pose problème.

Traçabilité et volume réaliste

- Ajoutez des champs **created_at** / **updated_at** sur les entités principales.
- Enrichissez le seed pour simuler un volume réaliste : 150 élèves ou plus, répartis sur plusieurs années, avec plusieurs examens, plusieurs tournois et un historique de compétences déjà débloquentes. Vérifiez que les endpoints de listing restent rapides à cette échelle (réappliquez la chasse au N+1 si un nouvel endpoint de la semaine en a besoin).

Documentation et livraison

- Collection Postman ou README avec exemples curl couvrant tous les endpoints de la semaine, y compris le mécanisme de login et le header X-User-Id, et les cas d'erreur de validation.
- README de démarrage complet : installation, variables d'environnement, commande de seed, commande de lancement, commande de tests.

Tests finaux

- Un test qui couvre les trois issues du passage d'année (promotion, redoublement, diplomation) sur des élèves aux profils différents.
- Un test qui vérifie qu'on ne peut pas clôturer deux fois la même année.

Critère de fin de journée

Un camarade d'un autre groupe peut cloner le repo, suivre le README sans aide, lancer la clôture de l'année sur le jeu de données de seed, et constater que les élèves sont correctement répartis entre promus, redoublants et diplômés — avec le détail de chaque décision consultable.

Livrables attendus (fin de jour 4)

- Code source de l'API, organisé comme jugé pertinent.
- Script de seed rejouable sans dupliquer ni casser les données, incluant un volume réaliste.
- Fichier de tests pytest exécutable en une seule commande, couvrant les scénarios métier des 4 jours.
- Collection Postman ou README avec exemples curl couvrant tous les endpoints.
- README de démarrage complet.
- PERFORMANCE.md ou section dédiée documentant la chasse au N+1 (avant/après).

Critères de réussite globaux

- Une personne extérieure lance le projet et l'utilise en suivant uniquement le README.
- Tous les endpoints répondent avec le bon code HTTP, y compris en cas d'erreur.
- La connexion simulée (login + X-User-Id) distingue correctement espace élève et espace admin sur l'ensemble des endpoints concernés.
- Les relations N-N (élève ↔ cours, élève ↔ compétence) sont vérifiables sans duplication.
- La clôture d'un examen déclenche correctement le déblocage des compétences associées, de façon idempotente.
- La clôture d'un tournoi désigne un vainqueur cohérent et met à jour la réputation de sa maison.
- Le passage de fin d'année produit les trois issues possibles (promotion, redoublement, diplomation) de façon cohérente et vérifiable, et refuse un second jeu sur la même année.
- Les tests couvrent au moins un scénario métier réel par jour et passent tous.
- Le script de seed est idempotent.
- Le README documente une amélioration mesurée de performance (N+1).
- La validation des payloads rejette proprement les données invalides avec un message exploitable.